📝 ATA DE DESENVOLVIMENTO: PROGRESSO DA ARQUITETURA DE CÓDIGO

Estado Geral do Projeto: MVP Core Pronto & Compilável (Cérebro e Músculos Alinhados)

Ambiente de Desenvolvimento: .NET 8 LTS (Long Term Support) — Rigor e Estabilidade Industrial.

Até ao momento, isolámos com sucesso as regras sagradas da Nanopaint de qualquer dependência tecnológica pesada e erguemos a primeira fase da infraestrutura real. Eis o raio-x exato do que já foi codificado, revisto e trancado na tua solução:

1. 📁 CAMADA DE DOMÍNIO (`NanopaintHub.Domain`) — CONCLUÍDA

O coração físico e as regras imutáveis da fábrica estão modelados e livres de frameworks externos.

- **`Entity.cs`:** Classe abstrata base que garante identidade universal (`Guid`) e controlo de auditoria temporal (`CreatedAt`, `LastModified`) para todas as tabelas.
- **`SyncStatus.cs`:** O Enum que governa a máquina de estados offline-first (`LocalOnly`, `PendingInsert`, `PendingUpdate`, `Synced`, `ValidationError`).
- **`ProductCategory.cs`:** Entidade para a categorização dinâmica de tintas e vernizes especiais.
- **`Supplier.cs` & `SupplierComponent.cs`:** Modelagem estruturada de fornecedores e rastreabilidade química global (via número CAS) para automação futura de fichas de segurança (SDS).
- **`Product.cs`:** Entidade mestre que une a tinta à sua categoria (`CategoryId`) e blinda as propriedades industriais dinâmicas (Liquid, Mechanical, Functional) através de strings JSON nativas de alta performance.
- **`TestBatch.cs`:** Entidade do laboratório com a **Lógica de Domínio** embutida nativamente para transições automáticas de estado de sincronização (`MarkWithValidationError`, `MarkAsSynced`, `UpdateValues`).
- **`DocumentTemplate.cs`:** O contentor imutável dos layouts estruturais (HTML/CSS) desenhados pelo marketing, gerindo margens numéricas físicas locais (`TopMargin`, `BottomMargin`).

2. 📁 CAMADA DE APLICAÇÃO (`NanopaintHub.Application`) — CONCLUÍDA

O "Cérebro Lógico" do sistema, contendo as interfaces abstratas e os maestros que orquestram o negócio.

- **Contratos de Persistência (`Contracts/`):** Definição estrita das interfaces de acesso a dados `IProductRepository` e `ITemplateRepository`.
- **Contrato de Geração (`IDocumentGeneratorService.cs`):** Interface agnóstica universal para emissão unificada de documentos industriais baseados no produto (`GenerateTdsAsync` e `GenerateSdsAsync`).
- **Contrato de Transação (`IUnitOfWork.cs`):** Interface que remove o poder de gravação isolada dos repositórios, movendo o gatilho do commit para o nível do Caso de Uso.
- **`ProductDto.cs`:** Data Transfer Object imutável (`record`) para transporte seguro de coleções genéricas estruturadas para o ecrã.
- **Casos de Uso (`Features/`):** * `GenerateTdsUseCase.cs` (Maestro da Ficha Técnica).
    - `GenerateSdsUseCase.cs` (Maestro da Ficha de Segurança).
    - *Mecânica:* Ambos os casos recebem as abstrações via Injeção de Dependência, validam a existência física da tinta e do template, disparam a geração binária e devolvem o array de bytes (`byte[]`) sem saber como a base de dados ou o PDF funcionam na realidade.

3. 📁 CAMADA DE INFRAESTRUTURA DE DADOS (`NanopaintHub.Infrastructure.LocalData`) — CONCLUÍDA

O primeiro grande músculo do sistema operativo local.

- **`NanopaintHubDbContext.cs`:** A ponte do Entity Framework Core com o SQLite. Configurada via Fluent API (`OnModelCreating`) aplicando `DeleteBehavior.Restrict` na relação Produto-Categoria (integridade referencial de nível de produção) e valores padrão estritos para evitar nulos nos campos JSON.
- **Gerador de Design-Time (`NanopaintHubDbContextFactory.cs`):** Fábrica configurada para permitir a execução de ferramentas CLI do EF Core isoladas da interface gráfica.
- **Migrations & Base de Dados:** Executada e selada com sucesso a migração inicial (`InitialCreate`), gerando fisicamente o ficheiro **`nanopaint.db`** na raiz da tua infraestrutura de dados.
- **Repositórios Atómicos & Unit of Work:** Implementação real do `ProductRepository`, `TemplateRepository` e `UnitOfWork`. Os repositórios operam agora exclusivamente na memória RAM mapeada pelo contexto do EF Core, e o `UnitOfWork` assume centralizadamente o `SaveChangesAsync()` no método `CommitAsync()`.

4. 📁 CAMADA DE INFRAESTRUTURA DE DOCUMENTOS (`NanopaintHub.Infrastructure.Documents`) — CONCLUÍDA

O motor de renderização comercial com precisão ao nível do pixel.

- **Integração Puppeteer Sharp:** Criação do pipeline que aciona um navegador Chromium invisível (*headless*), injeta o HTML e exporta os dados milimetricamente baseados nas PdfOptions de formato (A4) e margens em Pixels gravadas no banco.
- **Correção de Runtime (Disposable Fetcher):** Corrigido o erro de compilação da nova versão do Puppeteer, removendo a declaração `using` do `BrowserFetcher`, estabilizando o download em runtime.
- **Integração Handlebars.NET:** O motor analisa os JSONs nativos do domínio do produto, converte-os em estruturas iteráveis e funde-os dinamicamente no HTML através de loops automáticos e limpos de código, salvaguardando o isolamento do C#.